WEEK 03

데이터 표현 및 조작

주요 학습 목표

- 문자열 / 리스트 / 세트 / 딕셔너리
- 인덱싱과 슬라이싱 활용
- 내장 함수 활용하기
- 공부 기록 실습

데이터 표현 및 조작

3주차

개요 및 학습목표

이번 주차에는 **데이터를 효율적으로 표현하고 조작**하는 방법을 학습합니다. 문자열, 리스트, 딕셔너리 등 파이썬의 핵심 자료형을 이해하고, 이를 활용해 실제 데이터를 처리하는 능력을 기릅니다.

핵심 개념 (Core Concepts)

- **문자열(String)**: 인덱싱, 슬라이싱, 함수 활용
- **리스트(List)**: 데이터 관리, 추가/삭제/정렬
- **세트 & 딕셔너리**: 중복 제거 및 키-값 매핑

실습 과제 (Practice)

- **시험 점수 평균내기**: 리스트와 내장함수(sum, len) 활용
- **민수의 공부 기록**: 인덱싱과 슬라이싱으로 데이터 분석
- 주석 작성법 및 ASCII 코드 변환 실습

주석(Comment) 작성법

소스코드에 붙이는 설명글

주석은 프로그램 실행에 영향을 주지 않으면서 코드의 목적, 입력값, 로직 등을 설명하기 위해 사용합니다. 파이썬에서는 '#' 기호로 주석을 시작합니다.

기호로 시작 (Python이 해석하지 않음)

코드의 목적, 로직, 입출력 설명

```
# 두 정수를 입력받아 합을 구한다
x = int(input("정수 1: ")) # 첫번째 입력
y = int(input("정수 2: ")) # 두번째 입력
print(x + y) # 결과 출력
```

파이썬 자료형 개요

데이터를 다루는 4가지 핵심 도구

파이썬은 다양한 형태의 데이터를 효율적으로 처리하기 위해 여러 내장 자료형을 제공합니다. 각 자료형의 특징과 용도를 이해하면 문제 상황에 맞는 최적의 데이터 구조를 선택할 수 있습니다.

문자열 (String)

문자들의 집합으로 따옴표('', "")로 감싸서 표현하며, 인덱싱과 슬라이싱이 가능합니다.

리스트 (List)

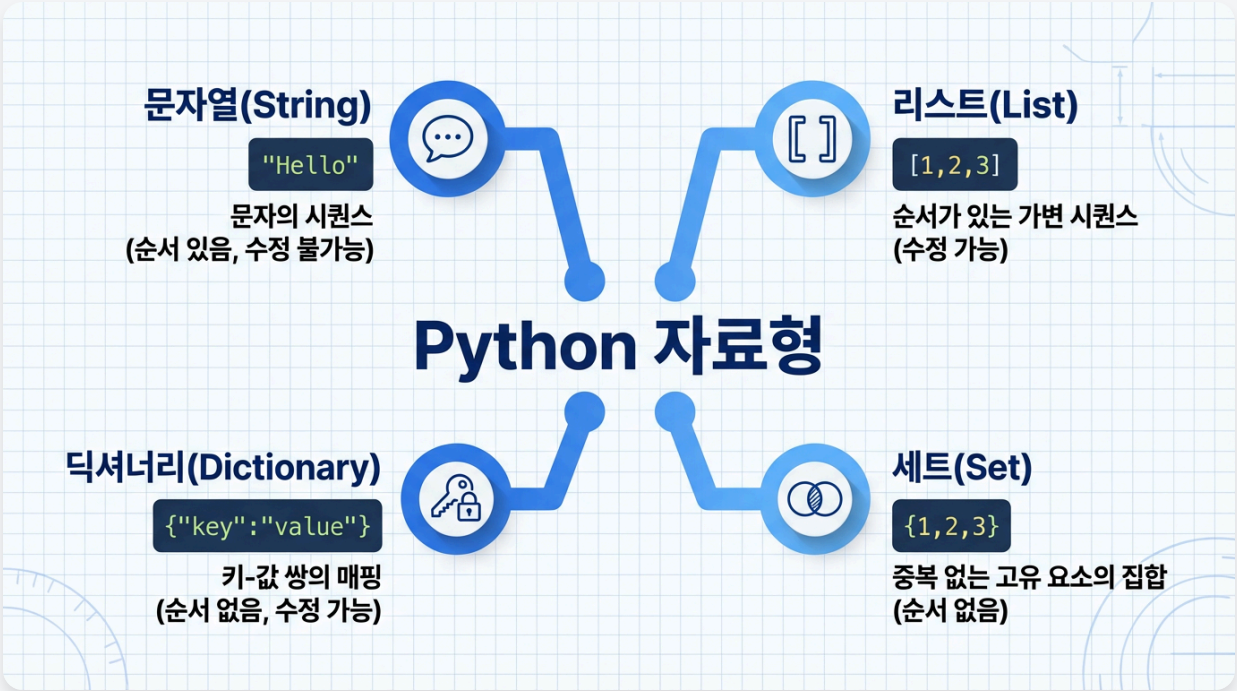
순서가 있는 데이터의 집합으로 대괄호([])를 사용하며, 값의 추가, 삭제, 수정이 자유롭습니다.

딕셔너리 (Dictionary)

키(Key)와 값(Value)의 쌍으로 이루어진 데이터로 중괄호({})를 사용하며 빠른 검색에 유리합니다.

세트 (Set)

순서가 없고 중복을 허용하지 않는 데이터의 집합으로 중괄호({})를 사용하며 집합 연산에 유용합니다.



스트링(String) 기본

파이썬에서 문자열은 텍스트 데이터를 처리하는 핵심 자료형입니다. 따옴표로 감싸서 표현하며, 다양한 연산이 가능합니다.

핵심 포인트

1. **작은따옴표(')** 또는 **큰따옴표(")**로 감싸서 문자열을 생성합니다.
2. **따옴표 3개(""" 또는 ''')**를 사용하면 여러 줄에 걸친 문자열을 만들 수 있습니다.
3. 주의: **'123'**은 숫자가 아니라 **문자열**입니다.

문자열 생성

단일 라인은 따옴표 한 개, 다중 라인은 따옴표 세 개를 사용합니다.

```
s1 = "Hello"
s2 = """동해물과 백두산이
마르고 닳도록"""
```

문자열 연산

+ 기호는 문자열을 연결하고, *** 기호**는 문자열을 반복합니다.

```
'a' + 'b' # 결과: 'ab'
'a' * 3 # 결과: 'aaa'
'123' * 2 # 결과: '123123' (숫자 연산 아님)
```

스트링 인덱싱과 슬라이싱

문자열 조작 기초 (p.41)

문자열의 특정 위치 문자를 참조하거나(`Indexing`), 일부분을 추출하는(`Slicing`) 방법을 학습합니다.

인덱스는 0부터 시작하며, 음수를 사용하여 뒤에서부터 접근할 수도 있습니다.

01

1. 인덱싱 (Indexing)

`s[0]` (첫 글자), `s[-1]` (마지막 글자)

02

2. 슬라이싱 (Slicing)

`s[1:3]` (1부터 2까지)
`s[:2]` (처음부터 1까지)

03

3. 길이 확인 (Length)

`len(s)`
문자열의 전체 길이 반환

문자열 함수: len(), in

문자열 길이와 포함 여부

문자열의 길이를 확인하거나 특정 문자가 포함되어 있는지 검사할 때 사용하는 유용한 기능들입니다. 데이터 처리 시 빈번하게 활용됩니다.

01.
len('abc') → 3
(Length)

02.
'백두산' in a
(Membership)

03.
부분문자열
탐색

ASCII 변환 ord(), chr()

컴퓨터는 문자를 숫자로 저장합니다. **ord()** 함수는 문자를 아스키(ASCII) 코드로, **chr()** 함수는 아스키 코드를 다시 문자로 변환합니다.

문자→숫자

ord('A')
↓
65

숫자→문자

chr(65)
↓
'A'

응용

이름 첫 글자를 숫자로 변환하여 표현하기

</> Python 예제

```python
# 문자 → 아스키 코드
print(ord('A')) # 65
print(ord('a')) # 97

# 아스키 코드 → 문자
print(chr(65)) # A
print(chr(97)) # a
```

자료구조 기초

리스트(List): 데이터의 묶음

리스트는 여러 개의 데이터를 순서대로 저장하는 **시퀀스 자료형**입니다. 하나의 변수에 숫자, 문자열 등 다양한 타입의 데이터를 담아 효율적으로 관리할 수 있는 파이썬의 가장 강력한 도구 중 하나입니다.

생성과 접근

- 대괄호 `[]`를 사용하여 생성
- 예: `scores = [78, 85, 92]`
- **인덱싱**: `s[0]` (첫 번째 요소)
- **슬라이싱**: `s[1:3]` (부분 추출)

가변 자료구조 (Mutable)

- 생성 후 내용의 **수정, 추가, 삭제** 가능
- `append()`: 리스트 끝에 요소 추가
- `del` 또는 `remove()`: 요소 삭제
- `sort()`: 정렬 기능 제공

리스트 활용 예제

리스트의 주요 기능 실습

리스트는 파이썬에서 가장 강력한 자료구조 중 하나입니다. 데이터를 생성하고 추가, 삭제하며 다양한 내장 함수를 통해 통계 처리를 수행할 수 있습니다.

1) 생성 → append() 추가

2) 삭제(del, remove) → 반복 처리

3) 합계(sum), 최대/최소, 정렬(sort)

세트(Set) 개념 이해

중복을 허용하지 않는 집합

세트(Set)는 순서가 없고 중복된 데이터를 허용하지 않는 자료형입니다. 리스트의 중복을 제거하거나, 교집합·합집합 같은 집합 연산을 수행할 때 매우 유용하게 사용됩니다.

01

중복 제거

s = set([1,2,3,3])
→ {1, 2, 3}

02

집합 연산

교집합(&), 합집합(|)
차집합(-)

03

응용 예제

데이터 필터링
고유값 추출

딕셔너리(Dictionary) 개념

Key:Value 매핑

`{Key: Value}`

딕셔너리는 키(Key)와 값(Value)의 쌍으로 데이터를 저장합니다. 순서가 없는 집합이며, 키를 통해 값을 빠르게 찾을 수 있는 해시 구조입니다.

데이터 접근

`d[key]`

리스트의 숫자 인덱스 대신 고유한 **키(Key)**를 사용하여 값에 접근합니다. 존재하지 않는 키를 호출하면 KeyError가 발생하므로 주의해야 합니다.

수정/추가/삭제

`d[key] = val`

새로운 키에 값을 할당하면 항목이 **추가**되고, 기존 키에 할당하면 값이 **수정**됩니다. **del d[key]** 명령어로 특정 항목을 삭제할 수 있습니다.

```
score = {'kor': 90, 'eng': 85}
print(score['kor'])   → 90 (키 'kor'의 값 출력)
```

실습: 시험점수 평균 계산

3주차: 데이터 표현 및 조작 (p.48)

리스트를 활용하여 5과목의 시험 점수를 저장하고 평균을 계산해 봅니다.

- **데이터:** 국어(78), 영어(85), 수학(92), 화학(66), 물리(88)
- **핵심 함수:** `sum()` 합계, `len()` 길이

01

1. 리스트 정의

```
scores = [78, 85, 92, 66, 88]
```

02

2. 합계 계산

```
total = sum(scores)
# 모든 원소의 합
```

03

3. 평균 계산

```
avg = total / len(scores)
print(avg)
```

종합실습: 민수의 공부 기록

리스트 활용 데이터 분석

민수의 과목별 공부시간 기록을 리스트로 정의하고, 인덱싱과 슬라이싱을 통해 전공과 교양 과목의 공부 시간을 각각 계산해 봅니다. 리스트 조작 능력을 종합적으로 점검하는 실습입니다.

```
# 데이터 구조 예시
study_hours = [2, 3, 1, 3]
# 과목 순서: 고급프로그래밍, 확률과통계, 철학, 컴퓨팅사고
```

1. 리스트 정의

study_hours = [2, 3, 1, 3]

2. 인덱싱 & 슬라이싱

전공: study_hours[:2] / 교양: study_hours[2:]

3. 합계 계산 (sum)

전공: 5시간, 교양: 4시간 → 총 9시간

3주차 실습 종합 정리

이번 주차에서는 문자열과 리스트, 딕셔너리, 세트 등 파이썬의 핵심 자료형을 다루었습니다.

데이터의 구조를 이해하고 인덱싱과 슬라이싱을 통해 데이터를 자유롭게 조작하는 방법을 실습했습니다.

문자열 조작

문자열 **연산**과
인덱싱, **슬라이싱**을 통해
텍스트 데이터를
효율적으로 처리

자료구조 활용

리스트의 순서,
세트의 중복 제거,
딕셔너리의 키-값 매핑
특성 이해

데이터 분석

시험 점수 **평균 계산**과
민수의 공부 기록 분석을 통한
실전 데이터 처리 경험

3주차
요약 및 예고

문자열, 리스트, 딕셔너리, 세트 등 다양한 데이터 표현 방식과 인덱싱, 슬라이싱을 통한 조작 방법을 익혔습니다. 평균 계산과 공부 기록 분석 실습으로 활용 능력을 높였습니다.

핵심 포인트 & 다음 주차:

01

데이터 조작 숙달:
인덱싱/슬라이싱

02

리스트 활용 능력:
생성/수정/연산

NEXT

문제 정의:
2진수 변환, 큐브